

Bilan mi-parcours — Structures de données en Rust

Projet BE — Phase 1

Maximilien Larroche 22301310

1. Introduction

Ce rapport présente le bilan de la première phase du projet, dont l'objectif est d'implémenter plusieurs structures de données en Rust en respectant un trait commun, puis d'évaluer leurs performances. Cette phase s'est concentrée sur l'apprentissage du langage Rust et l'implémentation de trois structures : l'arbre binaire de recherche (BST), un Hash-Set et un BTreeSet.

2. Implémentation

2.1. Apprentissage de Rust

L'apprentissage du langage Rust a constitué la partie la plus chronophage de cette phase. Les ressources utilisées sont le Rust Book [1] ainsi que des vidéos en français [2][3][4]. Les principales difficultés rencontrées concernent le système d'ownership, les types mutables, et la gestion des références.

2.2. Trait commun

Un trait `StructureDonnee` a été défini pour unifier les opérations de base sur toute structure de données :

```
pub trait StructureDonnee {
    fn new() -> Self;
    fn add(&mut self, value: i32);
    fn remove(&mut self, value: i32);
    fn there_is(&self, value: i32) -> bool;
    fn map(&self, f: impl Fn(i32) -> i32 + Copy) -> Self;
    fn iter(&self) -> Box<dyn Iterator<Item = i32> + '_>;
    fn fragmenter(self, taille_max: usize) -> Vec<Self>
    where
        Self: Sized + Clone;
}
```

Les opérations ensemblistes (union, intersection, différence, différence symétrique) ont été implémentées directement dans le trait à partir de `add`, `remove`, `iter` et `there_is`. Ce choix évite de les ré-implémenter pour chaque structure et permet d'ajouter facilement de nouvelles structures.

La fonction `delete` initialement prévue a été supprimée car Rust implémente le trait `Drop` qui libère la mémoire automatiquement à la sortie du scope.

2.3. Arbre binaire de recherche

L'arbre binaire de recherche a été choisi comme première structure car il est bien maîtrisé. La structure est définie ainsi :

```
pub struct Arbre {
    pub taille: usize,
    pub root: Option<Node>,
}
pub struct Node {
    pub value: i32,
    pub left: Option<Box<Self>>,
    pub right: Option<Box<Self>>,
}
```

L'utilisation de `Box<Node>` est nécessaire car Rust doit connaître la taille de chaque type à la compilation. Un type récursif direct aurait une taille infinie. `Option` permet de représenter l'absence d'enfant (`None`).

La suppression d'un nœud gère trois cas : nœud feuille (suppression directe), nœud avec un seul enfant (remplacement par l'enfant), et nœud avec deux enfants (remplacement par le successeur infixe — le nœud le plus à gauche du sous-arbre droit).

Un itérateur en profondeur (DFS) a été implémenté via une pile explicite pour parcourir l'arbre sans récursion, évitant les débordements de pile sur les grands arbres.

2.4. HashSet et BTreeSet

Pour permettre la comparaison, deux structures de la bibliothèque standard ont été wrappées via le pattern `newtype` afin de respecter la règle orpheline de Rust — qui interdit d'implémenter un trait extérieur sur un type extérieur :

```
pub struct MonHashSet(pub HashSet<i32>);
pub struct MonBTreeSet(pub BTreeSet<i32>);
```

Le `HashSet` utilise une table de hachage offrant des opérations en $O(1)$ en moyenne, mais sans ordre garanti. Le `BTreeSet` est un arbre B équilibré de la bibliothèque standard offrant des opérations en $O(\log n)$ avec un ordre trié garanti, comparable à un BST optimisé.

2.5. Opérations ensemblistes

Les quatre opérations ont été implémentées dans le trait en modifiant la structure appelante (`&mut self`) plutôt qu'en créant une nouvelle structure, afin de minimiser les allocations mémoire :

- **Union** : insertion de tous les éléments de `other` dans `self`
- **Intersection** : suppression des éléments de `self` absents de `other`
- **Différence** : suppression des éléments de `self` présents dans `other`
- **Différence symétrique** : collecte d'abord les éléments communs, puis suppression dans les deux structures pour éviter les conflits d'emprunt du borrow checker

3. Vérification fonctionnelle

La vérification s'est faite par tests manuels dans le main avec `println!`, en vérifiant le comportement dans les cas limites : arbre vide, nœud feuille, nœud avec un ou deux enfants, suppression de la racine.

Des fonctions de test unitaires avec `#[test]` n'ont pas été implémentées par manque de temps. Elles sont prévues pour la phase suivante et permettront à toute nouvelle structure implémentant le trait de passer les mêmes tests automatiquement.

4. Protocole expérimental

Les benchmarks ont été réalisés avec la crate `Criterion` [5] (version 0.5), qui effectue des mesures statistiques avec intervalle de confiance à 95%.

Environnement : Windows 11, processeur `x86_64`, avec uniquement VS Code ouvert pendant les mesures.

Paramètres : 50 échantillons par benchmark, 15 secondes de mesure, phase de chauffe de 3 secondes (valeur par défaut `Criterion`). Les structures sont remplies avec des valeurs aléatoires (crate `rand`) pour éviter la dégénérescence de l'arbre en liste chaînée lors des insertions ordonnées.

Les benchmarks ont été généralisés via une fonction générique contrainte par le trait, ce qui permet de comparer toutes les structures sans dupliquer le code :

```
fn bench_generique<T: StructureDonnee>(
    c: &mut Criterion, nom: &str
) { ... }
```

5. Résultats

Les benchmarks ont été réalisés sur des structures de 10 000 éléments. Les temps indiqués sont les médianes des 50 mesures.

Opération	BST	HashSet	BTreeSet
add 10 000	1.05 ms	287 µs	284 µs
remove 10 000	1.35 ms	413 µs	413 µs
find	9.0 ns	7.3 ns	7.3 ns
Union	1.14 ms	145 µs	143 µs
Différence	1.35 ms	269 µs	270 µs
Diff. symétrique	33.4 ns	39.0 ns	39.6 ns
Intersection	1.14 ms	145 µs	147 µs
Fragmenter	2.30 ms	109 µs	109 µs

Tableau 1. – Résultats des benchmarks — 10 000 éléments

Les résultats montrent que le `HashSet` et le `BTreeSet` sont significativement plus rapides que le `BST` maison sur la quasi-totalité des opérations — de 3x à 20x selon l'opération. Cela s'explique par les optimisations de la bibliothèque standard (cache-friendly, équilibrage automatique) et par le fait que le `BST` maison n'est pas équilibré.

La différence symétrique fait exception : le `BST` est plus rapide (33 ns vs 39 ns) car l'implémentation profite de la pro-

priété d'ordre du `BST` pour collecter les éléments communs plus efficacement.

La recherche (`find`) est proche pour les trois structures (7-9 ns), ce qui confirme la complexité $O(\log n)$ théorique du `BST` et du `BTreeSet`, et $O(1)$ du `HashSet`.

Le `BST` maison souffre probablement d'un déséquilibre résiduel malgré l'insertion aléatoire, ce qui dégrade ses performances par rapport au `BTreeSet` de la bibliothèque standard qui garantit l'équilibrage.

6. Conclusion

Cette première phase a permis de poser les bases du projet : maîtrise du langage Rust, définition d'un trait commun, implémentation complète d'un `BST` avec les opérations ensemblistes, et comparaison avec deux structures de la bibliothèque standard (`HashSet` et `BTreeSet`).

Les résultats montrent clairement l'impact des optimisations de la bibliothèque standard sur les performances. Pour la phase suivante, les priorités sont : l'implémentation de tests unitaires automatisés, l'ajout d'un arbre AVL pour comparer avec le `BST` maison, et la sérialisation/désérialisation.

7. Références

- [1] Klabnik S., Nichols C. *The Rust Programming Language*, 2023. <https://doc.rust-lang.org/book/>
- [2] *Apprendre le RUST partie #1 FR*, YouTube. https://www.youtube.com/watch?v=mZasv3__A9k&list=PLrT8DrHsxZTiiAj96QukmAdedfRMsIPN5
- [3] *Apprendre le RUST partie #2 FR*, YouTube. <https://www.youtube.com/watch?v=wgjw5lGv-EI&list=PLrT8DrHsxZTiiAj96QukmAdedfRMsIPN5&index=2>
- [4] *Apprendre le RUST partie #3 FR*, YouTube. <https://www.youtube.com/watch?v=3kBk3sjREOM&list=PLrT8DrHsxZTiiAj96QukmAdedfRMsIPN5&index=3>
- [5] Heisler B. *Criterion.rs — Statistics-driven benchmarking*, 2024. <https://bheisler.github.io/criterion.rs/book/>
- [6] Dépôt source : https://github.com/Artelien/projet_be